Docker
для Machine Learning

План занятия

Теория

1. Зачем Docker в ML?
2. Архитектура Docker (схема)
3. Контейнеры vs ВМ (сравнение)
4. Образы, слои, реестры
5. Dockerfile — синтаксис и ML-примеры
6. Команды Docker (cheat sheet)
7. Volumes и сеть
8. Docker Compose
9. GPU: NVIDIA Container Toolkit
10. Best Practices для ML
11. Dev Containers в VSCode

Практика

1. Первый контейнер
2. FastAPI веб-сервер в Docker
3. Client → Server архитектура
4. ML-пайплайн: данные → обучение → инференс
5. LogisticRegression в контейнере
6. Docker Compose: API + модель
7. Dev Container в VSCode
8. Создание своего .devcontainer

Зачем Docker в ML?

Воспроизводимость

Единая среда: Python, CUDA, библиотеки — работает одинаково на любой машине

Быстрый деплой

Модель + API → Docker image → запуск в облаке за секунды

Версионность

Каждый эксперимент — свой тег образа с фиксированными зависимостями

Масштабирование

Kubernetes / Docker Swarm — оркестрация обучения на кластере GPU

Командная работа

docker pull → у всех одинаковая среда, не нужно «а у меня работает»

Изоляция

Конфликтующие версии CUDA / Python / библиотек в разных контейнерах

Архитектура Docker

DOCKER CLIENT

`docker build`	Сборка образа
`docker run`	Запуск контейнера
`docker pull`	Скачать образ
`docker push`	Загрузить образ
`docker compose`	Multi-сервис

REST API →

DOCKER DAEMON (dockerd)

Images
Слоистая FS
Tags, Layers

Containers
Процессы
Namespaces

Volumes
Persistent
Data

Networks
Bridge, Host
Overlay

pull/push →

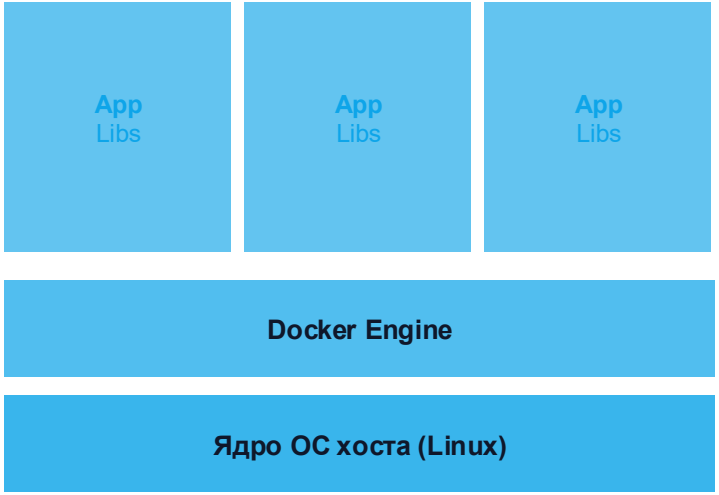
REGISTRY

Docker Hub	Public images
NGC (NVIDIA)	GPU/ML images
GHCR	GitHub packages
AWS ECR	Amazon private

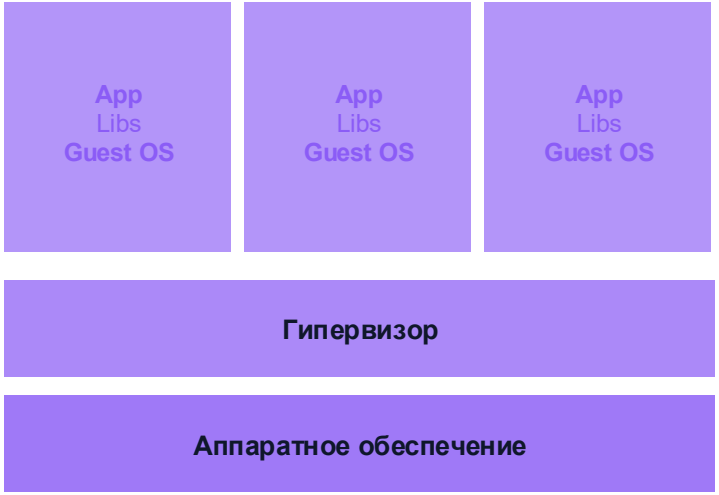
Host OS Kernel: namespaces • cgroups • UnionFS (OverlayFS) • seccomp • capabilities

Контейнеры vs Виртуальные машины

Контейнеры



Виртуальные машины



Запуск

Секунды vs Минуты

Размер

Мегабайты vs Гигабайты

Изоляция

Процесс vs Полная ОС

Overhead

Минимальный vs Значительный

Слои образа Docker

Каждая инструкция Dockerfile создаёт новый read-only слой

`Container Layer (R/W)`	← частые изменения	~variable	Запись: логи, модели, temp файлы
`CMD python train.py`		0 B	Точка входа — не создаёт данных
`COPY . .`		~5 MB	Код проекта — меняется часто ↑
`RUN pip install -r req.txt`		~2 GB	Зависимости Python — кэшируется!
`COPY requirements.txt .`		~1 KB	Только файл зависимостей
`RUN apt-get install python3`		~300 MB	Системные пакеты + очистка
`FROM nvidia/cuda:12.2`	← редкие изменения	~3.5 GB	Базовый образ NVIDIA

R/W

Image (R/O

Совет: ставьте часто меняющиеся инструкции в конец — Docker переиспользует кэш верхних слоёв

Ключевые понятия

Dockerfile

Рецепт: текстовый файл с инструкциями сборки. FROM, RUN, COPY, CMD

Image (Образ)

Неизменяемый шаблон. Слоистая FS. Можно тегировать: myapp:v1.0

Container

Запущенный образ. Изолированный процесс с PID 1 внутри namespace

Registry

Хранилище образов: Docker Hub, NGC, GHCR. docker push / pull

Dockerfile — Синтаксис

Инструкция	Назначение	Пример
`FROM`	Базовый образ	`FROM python:3.11-slim`
`RUN`	Выполнить команду	`RUN pip install torch`
`COPY`	Скопировать файлы	`COPY . /app`
`ADD`	Копирование + архивы/URL	`ADD data.tar.gz /data`
`WORKDIR`	Рабочая директория	`WORKDIR /app`
`ENV`	Переменная среды	`ENV PYTHONPATH=/app`
`EXPOSE`	Объявить порт	`EXPOSE 8000`
`CMD`	Команда по умолчанию	`CMD ["python", "app.py"]`
`ENTRYPOINT`	Фиксированная точка входа	`ENTRYPOINT ["python"]`
`ARG`	Build-time переменная	`ARG CUDA_VER=12.2`

Dockerfile для ML-проекта

```dockerfile
FROM nvidia/cuda:12.2.0-runtime-ubuntu22.04

# System dependencies
RUN apt-get update && apt-get install -y \
    python3.11 python3-pip git && \
    rm -rf /var/lib/apt/lists/*

WORKDIR /app

# Dependencies first (cache-friendly)
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Project code (changes often)
COPY . .

EXPOSE 8000
CMD ["python3", "train.py"]
```

- NVIDIA CUDA + Ubuntu базовый образ для GPU
- Системные зависимости + очистка кэша apt
- requirements.txt отдельно → слой кэшируется!
- Код копируется последним → частые изменения не инвалидируют кэш
- Точка входа обучения

Основные команды Docker

Сборка и запуск

`docker build -t app .`

Собрать образ

`docker run -it app bash`

Запустить интерактивно

`docker run -d -p 8000:8000 app`

Запуск в фоне, порт

`docker run --gpus all app`

С GPU

Управление

`docker ps / docker ps -a`

Контейнеры

`docker images`

Список образов

`docker logs <id>`

Логи

`docker exec -it <id> bash`

Войти внутрь

Данные и реестр

`docker run -v ./data:/app/data`

Volume mount

`docker cp <id>:/app/model.pkl .`

Копировать файл

`docker push user/app:v1`

Загрузить в реестр

`docker pull pytorch/pytorch:2.1`

Скачать образ

Docker Volumes

Хост-машина (файловая система)

./data/

train.csv, test.csv, val.csv

./models/

logreg.pkl, scaler.pkl

./logs/

train.log, metrics.json

./config/

config.yaml, params.json

:ro

:rw

:rw

:ro

Docker Container

/app/data/

read-only

/app/models/

read-write

/app/logs/

read-write

/app/config/

read-only

```
docker run \
 -v data:/app/data:ro \
 -v models:/app/models \
 -v logs:/app/logs \
 -v
config:/app/config:ro \
 ml-train
```

Tips:

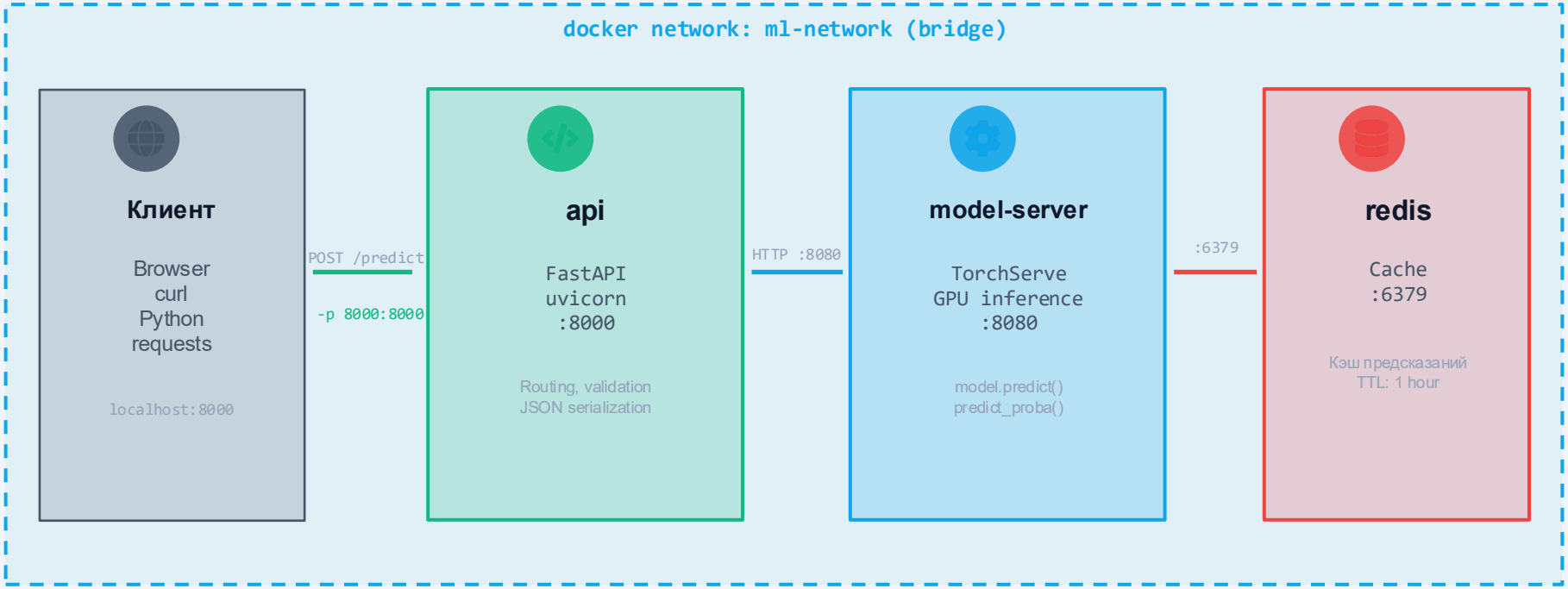
:ro — read-only

Данные НЕ в образ

Модели сохраняются на хосте навсегда

Docker Networking

Контейнеры общаются через виртуальные сети



Контейнеры обращаются друг к другу по имени сервиса: http://model-server:8080/predict • redis://redis:6379

Docker Compose для ML

Мульти-контейнерные ML-системы в одном файле

```yaml
# docker-compose.yml
services:
  api:
    build: ./api
    ports:
      - "8000:8000"
    depends_on:
      - model-server
    volumes:
      - ./logs:/app/logs

  model-server:
    build: ./model
    deploy:
      resources:
        reservations:
          devices:
            - driver: nvidia
              count: 1
    volumes:
      - ./models:/app/models

  mlflow:
    image: ghcr.io/mlflow/mlflow
    ports: ["5000:5000"]
```

API (FastAPI)

REST API для инференса, :8000

Model Server

GPU-контейнер, PyTorch модель

MLflow

Трекинг экспериментов, :5000

Redis (cache)

Кэш предсказаний

GPU: NVIDIA Container Toolkit

1

NVIDIA Driver на хосте

`nvidia-smi — проверить`

2

NVIDIA Container Toolkit

```
apt install nvidia-container-toolkit && systemctl restart docker
```

3

CUDA-образ

```
FROM nvidia/cuda:12.2.0-runtime-ubuntu22.04
```

4

Запуск с --gpus

```
docker run --gpus all  |  --gpus '"device=0,1"'
```

NGC (NVIDIA GPU Cloud) — готовые образы: nvcr.io/nvidia/pytorch:24.01-py3, nvcr.io/nvidia/tensorflow:24.01-tf2-py3

Best Practices для ML в Docker

Кэш слоёв

COPY requirements.txt ДО COPY . — зависимости кэшируются

Данные через volumes

Не COPY датасеты в образ! -v /data:/app/data

Multi-stage builds

Stage 1: обучение, Stage 2: только модель + инференс

.dockerignore

.git, __pycache__, *.pt, data/, .env, notebooks/

Фиксируйте версии

torch==2.1.0, numpy==1.25.0 — без версий нет воспроизводимости

Slim-образы

python:3.11-slim вместо python:3.11 — на ГБ меньше

Non-root user

RUN useradd -m appuser && USER appuser — безопасность

HEALTHCHECK

HEALTHCHECK CMD curl -f http://localhost:8000/health

Multi-Stage Build для ML

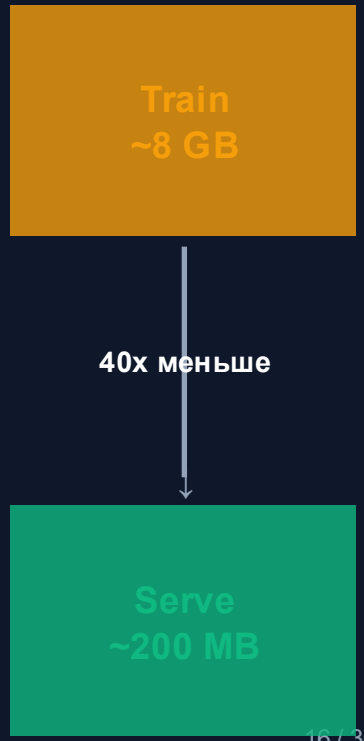
Разделяем обучение и инференс — итоговый образ в разы меньше

```
# Stage 1: Training
FROM nvidia/cuda:12.2.0 AS trainer
COPY requirements-train.txt .
RUN pip install -r requirements-train.txt
COPY . .
RUN python train.py --save /app/model.pkl
```

Stage 1: Train
~8 GB (CUDA + PyTorch)

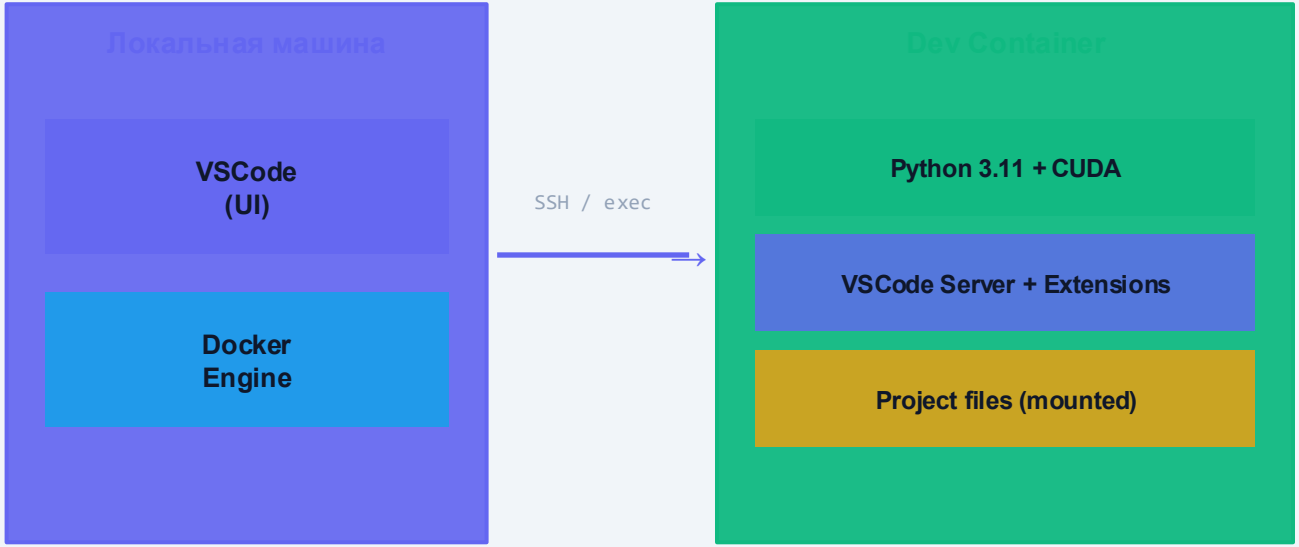
```
# Stage 2: Inference only
FROM python:3.11-slim AS serving
RUN pip install fastapi uvicorn scikit-learn
COPY --from=trainer /app/model.pkl .
COPY serve.py .
CMD ["uvicorn", "serve:app", "--host", "0.0.0.0"]
```

Stage 2: Serve
~200 MB (slim + model)



Dev Containers в VSCode

Разработка внутри контейнера — IDE подключается к Docker



Преимущества

- ✓ Одинаковая среда у всех
- ✓ Extensions внутри
- ✓ Полный IntelliSense
- ✓ Git интеграция
- ✓ Debugger работает
- ✓ Port forwarding

`.devcontainer/devcontainer.json`

```
{
  "name": "ML Dev",
  "image": "mcr.microsoft.com/devcontainers/python:3.11",
  "customizations": { "vscode": {
    "extensions": ["ms-python.python", "ms-toolsai.jupyter"]
  }},
  "postCreateCommand": "pip install -r requirements.txt"
}
```

Dev Container — конфигурация

```
// .devcontainer/devcontainer.json
{
  "name": "ML Workspace",
  "build": {
    "dockerfile": "Dockerfile",
    "context": ".."
  },
  "customizations": {
    "vscode": {
      "settings": {
        "python.defaultInterpreterPath": "/usr/bin/python3"
      },
      "extensions": [
        "ms-python.python",
        "ms-toolsai.jupyter",
        "ms-python.pylint"
      ]
    }
  },
  "forwardPorts": [8000, 8888],
  "postCreateCommand": "pip install -r requirements.txt",
  "mounts": [
    "source=${localWorkspaceFolder}/data,target=/data,type=bind"
  ]
}
```

- Свой Dockerfile для полного контроля над средой
- Python путь, linter настройки прямо в контейнере
- Расширения устанавливаются автоматически в контейнер
- Порты пробрасываются: Jupyter :8888, API :8000
- Монтирование датасета с хоста

Практика

FastAPI • ML Pipeline • Docker Compose • Dev Containers

Задание 1: Первый контейнер

1

1. **Проверяем установку Docker**

```
docker --version && docker run hello-world
```

2. **Запускаем Python-контейнер**

```
docker run -it python:3.11-slim bash
```

3. **Внутри: ставим numpy, проверяем**

```
pip install numpy && python -c "import numpy; print(numpy.__version__)"
```

4. **Одноразовый контейнер с командой**

```
docker run --rm python:3.11-slim python -c "print('Hello ML!')"
```

5. **Смотрим запущенные контейнеры**

```
docker ps -a && docker images
```

Задание 2: FastAPI веб-сервер

2

server.py

```python
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class PredictRequest(BaseModel):
    features: list[float]

@app.get("/health")
def health():
    return {"status": "ok"}

@app.post("/predict")
def predict(req: PredictRequest):
    # Здесь будет модель
    result = sum(req.features)
    return {"prediction": result}
```

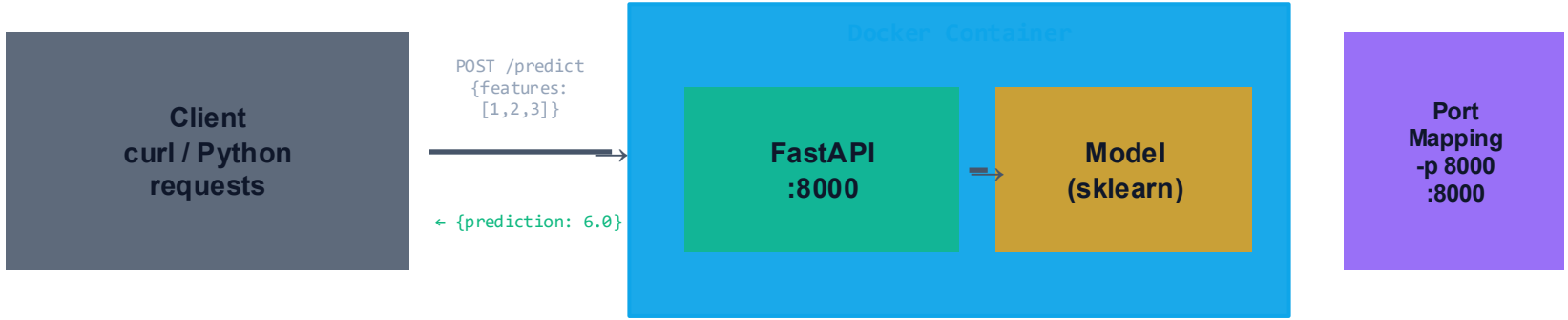
Dockerfile

```dockerfile
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir \
    -r requirements.txt

COPY server.py .

EXPOSE 8000
CMD ["uvicorn", "server:app", \
     "--host", "0.0.0.0", \
     "--port", "8000"]
```

requirements.txt: fastapi, uvicorn, pydantic

Client → Server в Docker

Архитектура и запуск



Сборка и запуск

```
docker build -t fastapi-ml .
```

```
docker run -d -p 8000:8000 fastapi-ml
```

Тестирование

```
curl localhost:8000/health
```

```
curl -X POST localhost:8000/predict
  -H 'Content-Type: application/json'
  -d '{"features": [1.0, 2.0, 3.0]}'
```

Задание 3: ML-пайплайн в Docker

3

Полный цикл: данные → обучение → сохранение → инференс

Данные
iris.csv
CSV / Parquet

→

Обработка
Pandas
train_test_split

→

Обучение
LogisticRegression
fit() → accuracy

→

Модель
joblib.dump()
logreg.pkl

→

Инференс
FastAPI
POST /predict

Структура проекта

```
ml-pipeline/
├── Dockerfile
├── requirements.txt
├── train.py          # Обучение LogReg
├── serve.py          # FastAPI инференс
├── data/
│   └── iris.csv
└── models/           # Сохранённые модели
```

requirements.txt

```
scikit-learn==1.3.2
pandas==2.1.4
numpy==1.26.2
joblib==1.3.2
fastapi==0.104.1
uvicorn==0.24.0
```

train.py — обучение LogisticRegression

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
import joblib, os

# 1. Загрузка данных
df = pd.read_csv("data/iris.csv")
X = df.drop("target", axis=1)
y = df["target"]

# 2. Разделение на train/test
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# 3. Обучение модели
model = LogisticRegression(max_iter=200)
model.fit(X_train, y_train)

# 4. Оценка
acc = accuracy_score(y_test, model.predict(X_test))
print(f"Accuracy: {acc:.4f}")

# 5. Сохранение модели
os.makedirs("models", exist_ok=True)
joblib.dump(model, "models/logreg.pkl")
print("Model saved to models/logreg.pkl")
```

Шаги:

1. Загрузка iris.csv из примонтированной папки data/
2. train_test_split — 80/20
3. LogisticRegression — простая, быстрая классификация
4. Метрика accuracy на тесте
5. joblib.dump — модель в файл для инференса

serve.py — инференс через FastAPI

```python
from fastapi import FastAPI
from pydantic import BaseModel
import joblib, numpy as np

# Загружаем модель при старте
model = joblib.load("models/logreg.pkl")

app = FastAPI(title='Iris Classifier')

class IrisInput(BaseModel):
    sepal_length: float
    sepal_width: float
    petal_length: float
    petal_width: float

@app.post("/predict")
def predict(iris: IrisInput):
    features = np.array([[
        iris.sepal_length, iris.sepal_width,
        iris.petal_length, iris.petal_width
    ]])
    pred = model.predict(features)[0]
    proba = model.predict_proba(features)[0]
    names = ["setosa", "versicolor", "virginica"]
    return {
        "class": names[pred],
        "probability": float(max(proba))
    }
```

- Модель загружается 1 раз при старте сервера
- Pydantic-схема валидирует входные данные автоматически
- predict + predict_proba для вероятностей
- JSON-ответ: класс + уверенность

Dockerfile для ML-пайплайна

```dockerfile
FROM python:3.11-slim

WORKDIR /app

# System deps for sklearn
RUN apt-get update && apt-get install -y \
    libgomp1 && rm -rf /var/lib/apt/lists/*

# Python deps (cached layer)
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Project code
COPY train.py serve.py ./

# Default: run training
CMD ["python", "train.py"]
```

Шаги запуска:

1. **Собрать образ**

```
docker build -t ml-pipeline .
```

2. **Обучение (train.py)**

```
docker run -v ./data:/app/data
  -v ./models:/app/models ml-pipeline
```

3. **Инференс (serve.py)**

```
docker run -p 8000:8000
  -v ./models:/app/models
  ml-pipeline uvicorn serve:app
  --host 0.0.0.0 --port 8000
```

4. **Тестирование**

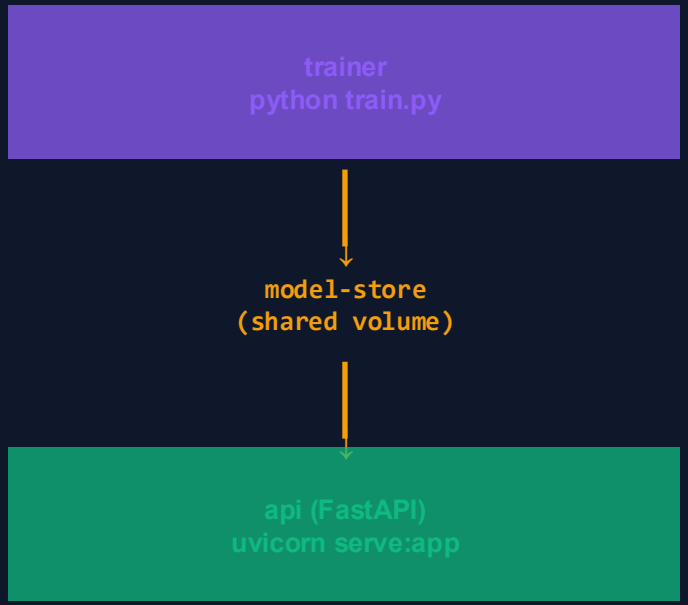
```
curl -X POST localhost:8000/predict
  -d '{"sepal_length":5.1, ...}'
```

Задание 4: Docker Compose — полная система

4

```yaml
# docker-compose.yml
services:
  trainer:
    build: .
    command: python train.py
    volumes:
      - ./data:/app/data
      - model-store:/app/models

  api:
    build: .
    command: uvicorn serve:app --host 0.0.0.0
    ports:
      - "8000:8000"
    volumes:
      - model-store:/app/models
    depends_on:
      trainer:
        condition: service_completed_successfully

volumes:
  model-store:
```

Архитектура:



depends_on:
condition:
service_completed_
successfully

Задание 5: Dev Container в VSCode

5

1. **Установите расширение Remote - Containers**

```
VSCode → Extensions → 'Dev Containers' → Install
```

2. **Создайте папку .devcontainer в проекте**

```
mkdir .devcontainer
```

3. **Создайте devcontainer.json (по шаблону с прошлого слайда)**

```
touch .devcontainer/devcontainer.json
```

4. **Создайте Dockerfile для dev-среды**

```
.devcontainer/Dockerfile
```

5. **Откройте проект в контейнере**

```
Ctrl+Shift+P → 'Dev Containers: Reopen in Container'
```

6. **Проверьте: Python, pip, extensions работают**

```
python --version && pip list && code --list-extensions
```

Dockerfile для Dev Container

```dockerfile
# .devcontainer/Dockerfile
FROM python:3.11-slim

# Dev tools
RUN apt-get update && apt-get install -y \
    git curl vim htop && \
    rm -rf /var/lib/apt/lists/*

# ML libraries
RUN pip install --no-cache-dir \
    scikit-learn pandas numpy \
    fastapi uvicorn joblib \
    jupyter matplotlib seaborn \
    pytest black ruff

# Non-root user
ARG USERNAME=vscode
RUN useradd -m $USERNAME
USER $USERNAME

WORKDIR /workspace
```

Что внутри:

ML фреймворки

sklearn, pandas, numpy, matplotlib

Web framework

FastAPI + Uvicorn

Jupyter

Notebook прямо в VSCode

Dev tools

git, black, ruff, pytest

Безопасность

Non-root user (vscode)